МИНИСТЕРСТВО ОБРАЗОВАНИЯ КУЗБАССА

ГПОУ «ЮРГИНСКИЙ ТЕХНОЛОГИЧЕСКИЙ КОЛЛЕДЖ»
ИМ. ПАВЛЮЧКОВА Г.А.
Отделение АИТ

РАЗРАБОТКА И ЗАПУСК МОДУЛЬНЫХ ТЕСТОВ

МЕТОДИЧЕСКИЕ УКАЗАНИЯ
ПО ПРАКТИЧЕКОЙ РАБОТЕ № 1

МДК.01.03 ТЕСТИРОВАНИЕ ИНФОРМАЦИОННЫХ РЕСУРСОВ

Специальность 09.02.09 Веб-разработка

Составитель
Соболева Д. А.

2025 г.

Цель: научиться разрабатывать и запускать модульные тесты, освоить написание корректных тестов для отдельных функций и методов, закрепить навыки выявления и исправления ошибок в тестах, подготовить основу для автоматизированной проверки корректности работы программного кода.

Краткая теория

Модульное тестирование — это проверка отдельных частей программы, таких как функции, методы или классы, на корректность их работы. Оно позволяет быстро выявлять ошибки и упрощает сопровождение кода, обеспечивая автоматическую проверку функциональности. Модульное тестирование повышает качество и надёжность программного обеспечения.

В Python для этого чаще всего используют фреймворки unittest и pytest. Первый является стандартным модулем, а второй более современный и удобный, поддерживает параметризацию тестов и формирует информативные отчёты. Для имитации внешних зависимостей применяют объекты mock.

При написании тестов важно избегать синтаксических и логических ошибок, проверять правильность условий, согласованность данных и корректность обработки исключений. Каждый тест должен быть независимым, его имя должно начинаться с test_, а для проверки результатов используют оператор assert.

Модульное тестирование позволяет не только быстро обнаруживать ошибки, но и улучшает структуру кода, делает его более понятным и поддерживаемым.

Задания:

1. Найдите и исправьте ошибку в тесте продемонстрированном на рисунке 1.

```
1   def subtract(a, b):
2       return a - b
3
4   def test_subtract():
5       assert subtract(10, 5) == 6
```

Рисунок 1 – Задание 1

2. Исправьте синтаксическую ошибку в тесте, продемонстрированном на рисунке 2.

```
1   def divide(a, b):
2       return a / b
3
4   def test_divide():
5       assert divide(10, 2) = 5
```

Рисунок 2 - Задание 2

3. Найдите логическую ошибку в тесте, продемонстрированном на рисунке 3.

```
1   def is_even(number):
2       return number % 2 != 0
3
4   def test_is_even():
5       assert is_even(4) == True
6       assert is_even(7) == False
```

Рисунок 3 - Задание 3

4. Исправьте ошибку при использовании PyTest, продемонстрированном на рисунке 4.

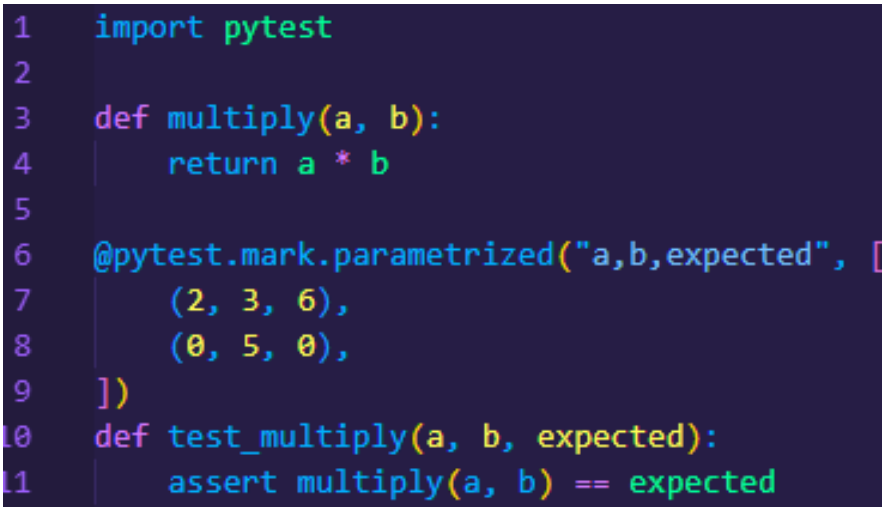


Рисунок 4 - Задание 4

5. Исправьте ошибку при тестировании функции, которая должна выбрасывать исключение, продемонстрированном на рисунке 5.

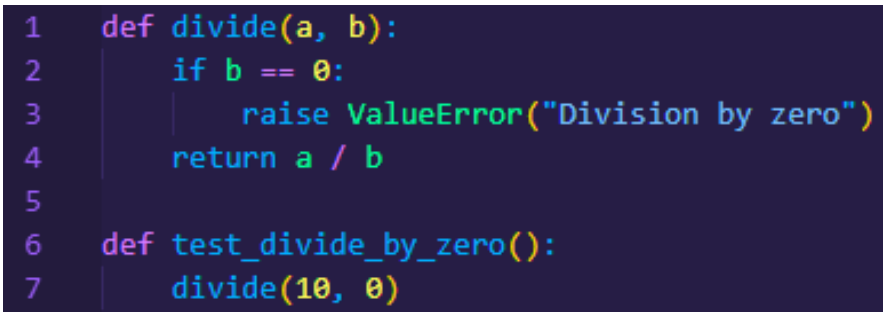


Рисунок 5 - Задание 5